UNIVERSITY OF EAST LONDON

School of Architecture, Computing & Engineering

Design and Implementation of a Web-Based Portal for a CVD Clinic

Using Enterprise Cloud Architecture CASE Tools

Case Study 4: Patients' Record for a CVD Clinic

CN6001 | Enterprise Architecture and Cloud Computing

2025 / 2026 — Spring Semester

GROUP MEMBERS

Hard Joshi
2512658
Scrum Master & Lead Developer

Jayrup Nakawala
2313621
Product Owner & Cardiovascular Consultant

Fatema Doctor
2604383
Frontend Developer & Nurse

Sangeet Kaur
2280454
QA, Database & Admin Staff

Submission Date: 1 May 2026

Table of Contents

1 Introduction . . . 1
1.1 Overview of the CVD Clinic . . . 1
1.2 Core Business Processes . . . 1
1.3 Proposed Digital Solution . . . 2
1.4 Scope of Work . . . 2
2 Business Process Model (BPMN) . . . 3
2.1 Diagram 1: Patient Admission and Form Completion . . . 3
2.2 Diagram 2: Appointment Booking Process . . . 4
2.3 Diagram 3: Form Processing and Data Flow . . . 4
2.4 Summary of BPMN Notation Usage . . . 5
3 Class Diagram and Entity Relationship Diagram . . . 6
3.1 Class Diagram . . . 6
3.1.1 Core Classes . . . 6
3.1.2 Class Relationships and Cardinalities . . . 7
3.2 Entity Relationship Diagram (ERD) . . . 8
3.2.1 UML to ERD Conversion . . . 8
3.2.2 Relationship Types . . . 9
4 Enterprise Cloud Architecture . . . 10
4.1 Architecture Overview . . . 10
4.2 On-Premise Infrastructure (Clinic) . . . 10
4.3 Off-Premise Cloud Architecture (AWS) . . . 11
4.3.1 VPC Structure . . . 11
4.3.2 Connectivity and Security Components . . . 12
4.4 Security and GDPR Compliance . . . 12
4.5 Justification for Cloud Adoption . . . 12
5 Prototype Implementation . . . 14
5.1 Technology Stack . . . 14
5.2 Architectural Strategy: Dual-SOA vs. Hybrid AOA/SOA . . . 14
5.3 Database Implementation . . . 15
5.4 Prototype Screens . . . 16
5.4.1 Screen 1: Patient Login / Registration . . . 17
5.4.2 Screen 2: Pre-Visit Form (Section 1) . . . 17
5.4.3 Screen 3: Staff Dashboard . . . 18
5.4.4 Screen 4: Examination Form (Section 2) . . . 19
5.4.5 Screen 5: Prescription Generation . . . 19
5.4.6 Screen 6: Appointment Booking . . . 20
5.5 Key Implementation Features . . . 20
6 Agile Project Management . . . 21
6.1 Team Roles . . . 21
6.2 Sprint Structure . . . 21

6.2.1 Sprint 1: Foundation (Weeks 1–2) . . . 21
6.2.2 Sprint 2: Patient Features (Weeks 3–4) . . . 22
6.2.3 Sprint 3: Staff Features (Weeks 5–6) . . . 22
6.2.4 Sprint 4: Prescription and Finalisation (Weeks 7–8) . . . 22
6.3 ScrumDesk Dashboard . . . 23
6.4 Tracking Metrics . . . 24
7 Individual Reflective Evaluation . . . 25
7.1 Individual Reflection: Hard Joshi (Scrum Master & Backend Architecture Lead) . . . 25
7.1.1 Personal Contribution & Challenges Encountered . . . 25
7.1.2 Application of Enterprise Architecture Concepts . . . 25
7.2 Individual Reflection: Jayrup Nakawala (Product Owner & Consultant) . . . 26
7.2.1 Personal Contribution & Challenges . . . 26
7.3 Individual Reflection: Fatema Doctor (Frontend Developer) . . . 26
7.3.1 Personal Contribution & Challenges . . . 26
7.4 Individual Reflection: Sangeet Kaur (QA & Database Engineer) . . . 27
7.4.1 Personal Contribution & Challenges . . . 27
7.5 Cloud Technology Analysis . . . 28
7.5.1 Analysis and Lab References . . . 29
7.6 Testing Conducted . . . 29
8 Group Conclusion . . . 31
8.1 Summary of Achievements . . . 31
8.2 Challenges and Solutions . . . 32
8.3 Future Improvements . . . 32
8.4 Final Reflection . . . 32
9 References . . . 33
10 Appendices . . . 34
10.1 Appendix A: Full Database Schema . . . 34
10.2 Appendix B: Additional Screenshots . . . 35
10.3 Appendix C: Test Evidence . . . 36
10.4 Appendix D: Revised BPMN Diagrams . . . 37

List of Figures

Figure 1 BPMN Diagram 1 — Patient Admission and Form Completion Process . . . 3
Figure 2 BPMN Diagram 2 — Appointment Booking Process . . . 4
Figure 3 BPMN Diagram 3 — Form Processing and Data Flow . . . 5
Figure 4 UML Class Diagram for the CVD Clinic Patient Record System (StarUML). . . . 6
Figure 5 Entity Relationship Diagram (ERD) for the CVD Clinic system (StarUML). . . . 8
Figure 6 Enterprise Cloud Architecture — On-Premise and AWS VPC Design (Draw.io). . . . 10
Figure 7 Architectural Evolution — Transition from Hybrid AOA/SOA Requirements to Implemented Dual-SOA Strategy. . . . 15

Figure 8 Patient Login and Registration Portal (SOA Interface). 17
Figure 9 Empty Pre-Visit Form. 17
Figure 10 Successful Submission. 17
Figure 11 Staff Login Gateway. 18
Figure 12 Admin Dashboard. 18
Figure 13 Nurse Dashboard. 18
Figure 14 Consultant Dashboard. 18
Figure 15 Examination Form (Section 2) with diagnosis and severity selected. 19
Figure 16 Final Unified Medical Record including the generated prescription. 19
Figure 17 Successful Appointment Booking interface. 20
Figure 18 Agile Sprint Board (Sprint 4) showing task distribution and completion status. . 23
Figure 19 Sprint 4 Burndown Chart showing actual vs. ideal progress with milestone annotations. 24
Figure 20 Appendix B.1: Incorrect Credentials 35
Figure 21 Appendix B.2: Duplicate Appointments 35
Figure 22 Appendix B.3: Form Validation on Section 2 36
Figure 23 Appendix C.1: Test Evidence - Successful vitals entry by Nursing Staff 36
Figure 24 Appendix C.2: Test Evidence - Medical record accurately reflecting updated vitals 37
Figure 25 Appendix D.1: Revised BPMN Diagram — Patient Admission and Form Completion Process. 38
Figure 26 Appendix D.2: Revised BPMN Diagram — Appointment Booking Process. 39
Figure 27 Appendix D.3: Revised BPMN Diagram — Form Processing and Data Flow. . . . 39

List of Tables

Table 1 Core business processes at the CVD Clinic. 1
Table 2 Summary of BPMN elements used across the three process models. 5
Table 3 Summary of classes, attributes, and operations in the CVD Clinic system. 7
Table 4 Entity table mapping from class diagram to ERD. 9
Table 5 AWS VPC subnet structure for the CVD Clinic system. 11
Table 6 AWS security and connectivity components. 12
Table 7 Technology stack used for the prototype implementation. 14
Table 8 Team roles and responsibilities within the Scrum framework. 21
Table 9 Comparison of AWS vs. Azure cloud platforms. 28
Table 10 Test plan with results for the CVD Clinic prototype. 30
Table 11 Key challenges encountered and their resolutions. 32

1 Introduction

1.1 Overview of the CVD Clinic

A group of cardiovascular (CVD) specialists operate a clinic located in West London, providing specialised care for patients with hypertension, hypotension, diabetes-related cardiovascular conditions, and general cardiovascular health monitoring. The clinic employs cardiovascular consultants, registered nurses, and administrative staff who collectively manage patient intake, vital signs recording, specialist consultations, and prescription management.

Currently, all patient data is collected through paper-based forms that patients fill in prior to examination. Each form typically consists of two sections: **Section 1**, completed by the patient (covering personal details, symptoms, medical history, and lifestyle factors), and **Section 2**, completed by the consultant following the clinical examination (including diagnosis, clinical notes, and treatment plan). This manual process has been identified as a key source of operational inefficiency, contributing to increased waiting times, data entry errors, and limited access to historical patient records.

1.2 Core Business Processes

The following table summarises the principal business processes currently in operation at the CVD Clinic:

Process	Description
Patient Registration	New patients provide personal and medical history details upon their first visit to the clinic.
Pre-Examination Forms	Patients complete Section 1 of the medical form — covering symptoms, current medications, allergies, and lifestyle factors — prior to seeing the consultant.
Vital Signs Recording	Nursing staff record BMI, blood pressure (systolic/diastolic), blood sugar levels, and heart rate before the consultation.
Specialist Consultation	The consultant reviews the patient's Section 1 data, conducts a physical examination, and completes Section 2 with diagnosis and clinical notes.
Prescription Generation	Following diagnosis, the consultant generates a prescription detailing medications, dosage, frequency, and instructions.
Appointment Booking	Administrative staff manage scheduling of initial and follow-up appointments, allocating consultant time slots.

Table 1: Core business processes at the CVD Clinic.

1.3 Proposed Digital Solution

This project proposes the design and implementation of a **web-based enterprise portal** to digitise the CVD Clinic's operations. The proposed system addresses the current pain points through the following capabilities:

- **Patient Self-Service Portal** — Enabling patients to register, complete pre-visit forms (Section 1) online, view their medical history, and book appointments remotely.
- **Staff Clinical Dashboard** — Providing consultants and nurses with a centralised interface to view patient data, record vital signs, complete clinical assessments (Section 2), and generate prescriptions.
- **Cloud-Based Infrastructure** — Hosting the system on AWS/Azure with a secure VPC architecture to ensure data availability, GDPR compliance, and disaster recovery.
- **Automated Workflows** — Implementing form validation with drop-down menus, auto-fill from historical data, and automated prescription generation to reduce manual errors.

1.4 Scope of Work

The scope of this coursework encompasses the full enterprise architecture lifecycle:

1. Business process modelling using **Bonita BPMN**
2. Class diagram and Entity Relationship Diagram (ERD) design using **StarUML**
3. Enterprise Cloud Architecture (ECA) design using **Draw.io**
4. Prototype web portal implementation with database backend
5. Agile project management using **ScrumDesk**
6. Individual reflective evaluations and group conclusion

2 Business Process Model (BPMN)

This section presents the Business Process Model and Notation (BPMN) diagrams created using **Bonita** software. Three process models have been developed to capture the key workflows of the CVD Clinic system. Each diagram identifies the participants (lanes/pools), tasks, gateways, and data objects involved.

2.1 Diagram 1: Patient Admission and Form Completion

The first BPMN diagram models the end-to-end patient admission process, from arrival at the clinic through to consultation and prescription receipt. The process involves three swim lanes: **Patient**, **Nurse/Admin Staff**, and **Consultant**.

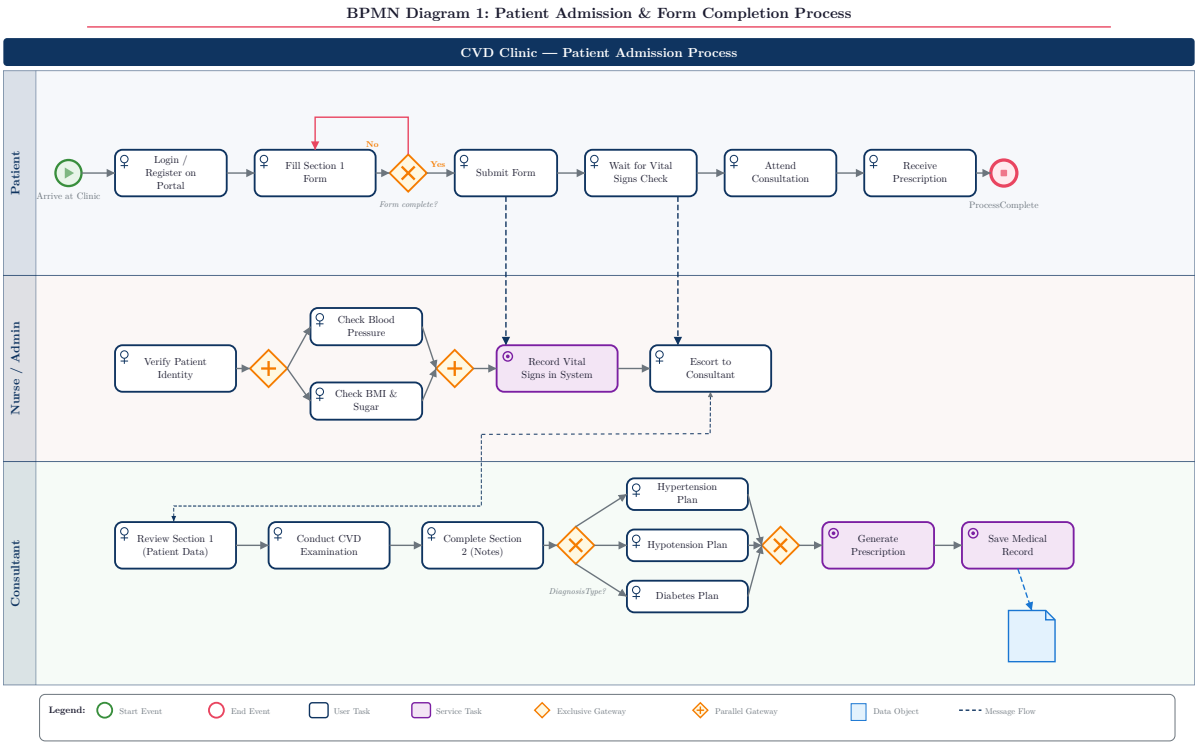

Figure 1: BPMN Diagram 1 — Patient Admission and Form Completion Process .

Process Description:

The enhanced BPMN model captures the end-to-end admission journey across four lanes: **Patient**, **Nurse/Admin Staff**, **Consultant**, and **System**. The process starts with patient arrival and portal login. A critical **correction loop** ensures that if the patient submits an incomplete Section 1, the system requests corrections and routes the flow back to the patient.

Following validation, the system triggers automated **service tasks** to auto-save the form and notify staff. Nursing staff then verify identity and perform **parallel vital signs checks** (simultaneous recording of BP, BMI, Blood Sugar, and Heart Rate) before recording them in the system. The consultant then reviews the unified data, conducts the examination, and selects

a diagnosis (Hypertension, Hypotension, or Diabetes) which branches into specific management plans. The process concludes with automated prescription generation, an optional follow-up scheduling step, and a final record save by the system.

Key BPMN Elements Used:

- **Correction Loop** — Gateways and flows to ensure data integrity in Section 1.
- **Service Tasks** — Automated actions (gear icon) for auto-save, notifications, and PDF generation.
- **Parallel Gateways (+)** — Modeling the simultaneous recording of multiple vital signs.
- **Data Objects** — Explicitly attached Patient Form, Medical Record, and Prescription artifacts.
- **Lanes** — Dedicated swimlanes for Patient, Nurse/Admin, Consultant, and System.

2.2 Diagram 2: Appointment Booking Process

The second BPMN diagram captures the appointment scheduling workflow involving the **Patient** and **Administrative Staff** lanes.

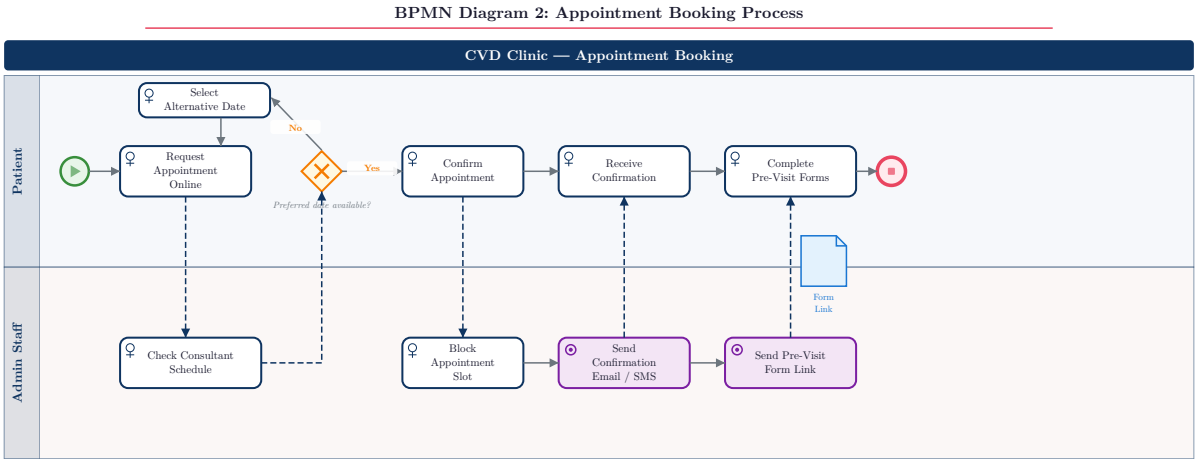

Figure 2: BPMN Diagram 2 — Appointment Booking Process .

Process Description:

This diagram clearly separates **Patient** and **Admin Staff** responsibilities. The patient requests an appointment online; if the date is unavailable, an **alternative date loop** returns the flow to the selection step. Once the admin blocks the slot and confirms, the system sends an **Appointment Confirmation** message and a **Pre-Visit Form Link** as explicit data artifacts. This process directly triggers the patient journey in Diagram 1.

2.3 Diagram 3: Form Processing and Data Flow

The third BPMN diagram illustrates how data flows between Section 1 (patient-completed) and Section 2 (consultant-completed) to create a comprehensive medical record.

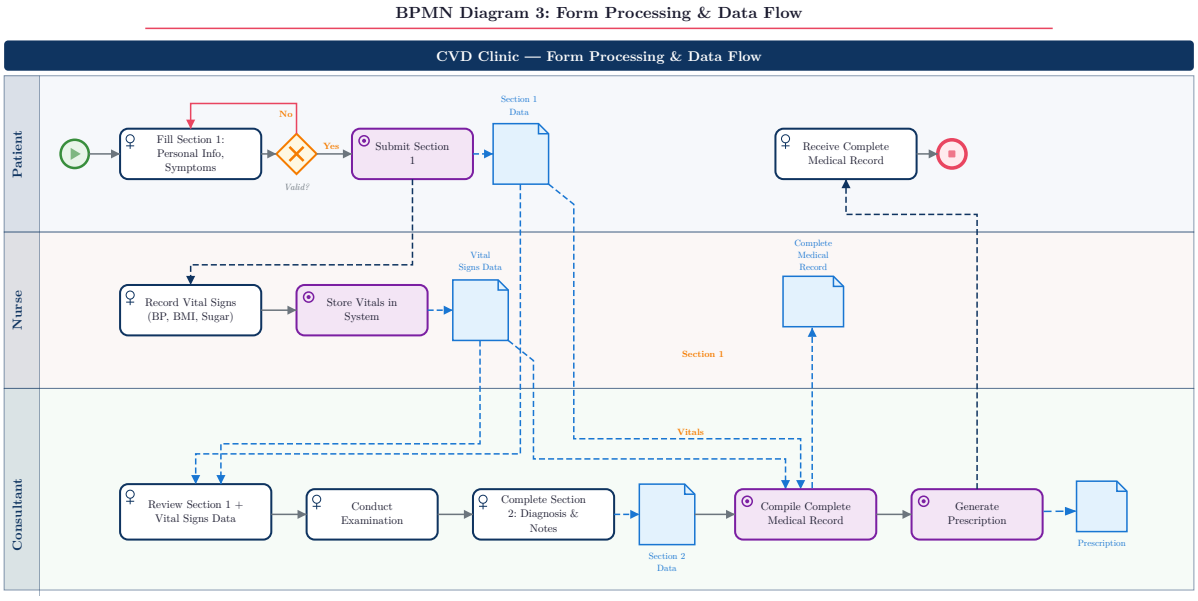

Figure 3: BPMN Diagram 3 — Form Processing and Data Flow .

Process Description:

Diagram 3 models the data lifecycle across the **Patient**, **Nurse**, **Consultant**, and **System** lanes. It highlights the **System lane** as the automated compiler of the medical record. Section 1 data, Vital Signs data, and Section 2 data are linked via associations to an automated **Compile Complete Medical Record** service task. This unified data then triggers the automated prescription generation service, completing the digital transformation of the paper-based form.

2.4 Summary of BPMN Notation Usage

BPMN Element	Application in CVD Clinic
Start/End Events	Patient arrives → Consultation complete
User Tasks	Verify Identity, Conduct Examination, Complete Section 2
Service Tasks	Auto-save, Send Notification, PDF Generation, Save Record
Exclusive Gateways	Section 1 Valid? Diagnosis Type? Follow-up Required?
Parallel Gateways	Simultaneous BP, BMI, Sugar, and Heart Rate checks
Data Objects	Patient Form, Medical Record, Prescription
Pools/Lanes	Patient, Nurse/Admin Staff, Consultant, System

Table 2: Summary of BPMN elements used across the three process models.

3 Class Diagram and Entity Relationship Diagram

This section presents the object-oriented class model and its corresponding Entity Relationship Diagram (ERD) for the CVD Clinic system, both designed using **StarUML**. The class diagram captures the structure, attributes, operations, and relationships between domain objects, while the ERD translates these into a relational database schema.

3.1 Class Diagram

The class diagram identifies nine core classes within the CVD Clinic domain. Each class includes its attributes (with types), operations (with parameters), and the associations/cardinalities between classes.

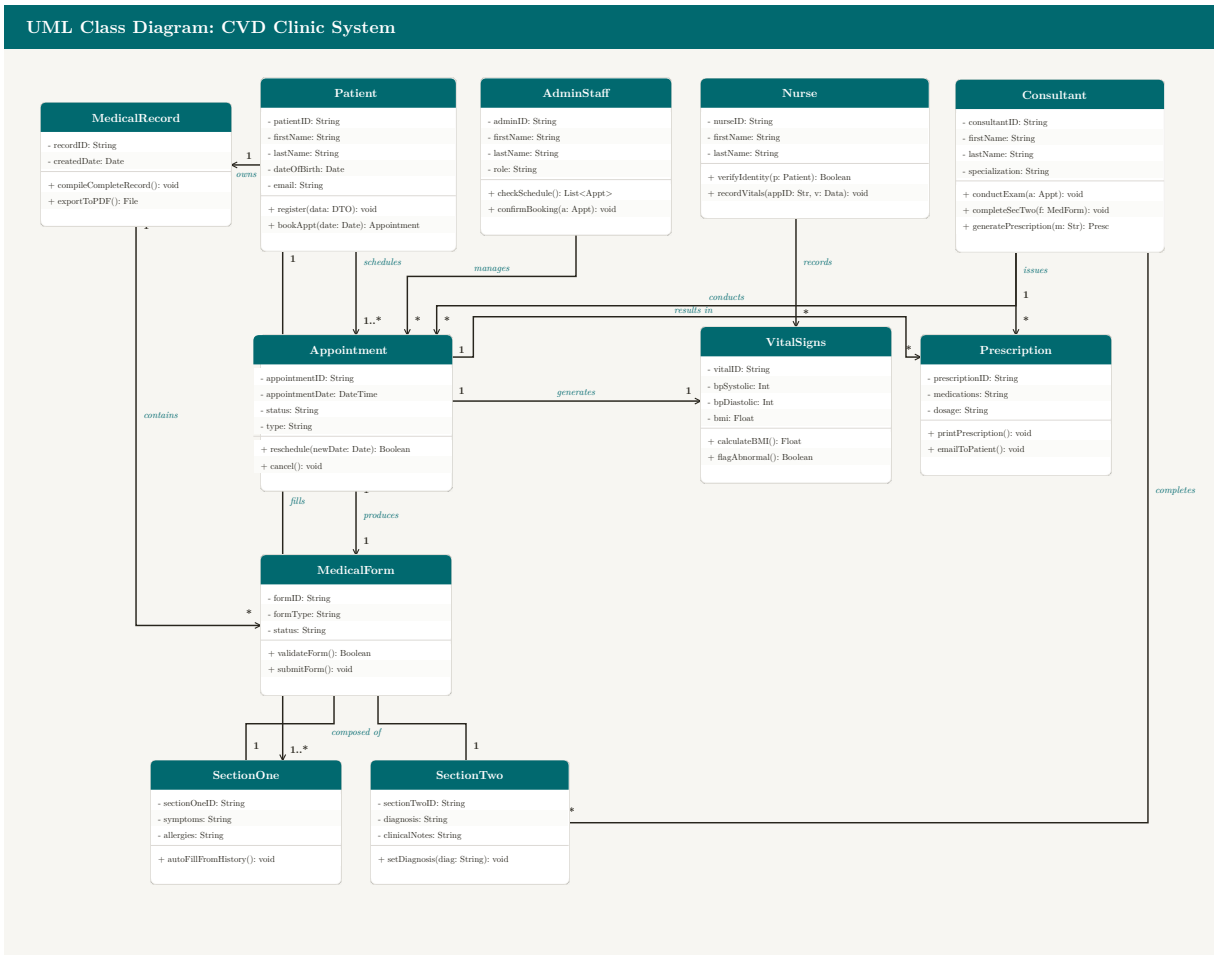

Figure 4: UML Class Diagram for the CVD Clinic Patient Record System (StarUML).

3.1.1 Core Classes

The following table summarises the nine classes, their key attributes, and primary operations:

Class	Key Attributes	Key Operations
Patient	patientID (PK), firstName, lastName, DOB, email, nhsNumber	register(), updateProfile(), viewHistory(), bookAppointment()
MedicalForm	formID (PK), patientID (FK), formType, status, dateCreated	submitForm(), editForm(), validateForm()
SectionOne	sectionOneID (PK), formID (FK), symptoms, medications, allergies	autoFillFromHistory(), validateRequiredFields()
SectionTwo	sectionTwoID (PK), formID (FK), consultantID (FK), diagnosis, severity	addClinicalNotes(), setDiagnosis()
Consultant	consultantID (PK), name, specialization, licenseNumber	examinePatient(), completeSectionTwo(), generatePrescription()
VitalSigns	vitalID (PK), patientID (FK), bpSystolic, bpDiastolic, bmi, bloodSugar	calculateBMI(), flagAbnormalValues()
Prescription	prescriptionID (PK), patientID (FK), consultantID (FK), medications, dosage	generatePrescription(), sendToPharmacy(), printPrescription()
Appointment	appointmentID (PK), patientID (FK), consultantID (FK), date, type, status	schedule(), reschedule(), cancel(), sendReminder()
MedicalRecord	recordID (PK), patientID (FK), formID (FK), vitalID (FK), prescriptionID (FK)	compileCompleteRecord(), exportToPDF()

Table 3: Summary of classes, attributes, and operations in the CVD Clinic system.

3.1.2 Class Relationships and Cardinalities

The following associations have been identified between the domain classes:

- **Patient** `1 --- *` **MedicalForm** — A patient may have multiple medical forms over time.
- **Patient** `1 --- *` **VitalSigns** — A patient has multiple vital sign recordings.
- **Patient** `1 --- *` **Prescription** — A patient may receive multiple prescriptions.
- **Patient** `1 --- *` **Appointment** — A patient can have multiple appointments.
- **Patient** `1 --- 1` **MedicalRecord** — Each patient has one comprehensive medical record.
- **MedicalForm** `1 --- 1` **SectionOne** — Each form has exactly one patient section.
- **MedicalForm** `1 --- 1` **SectionTwo** — Each form has exactly one consultant section.
- **Consultant** `1 --- *` **SectionTwo** — A consultant completes many Section 2 forms.
- **Consultant** `1 --- *` **Prescription** — A consultant issues many prescriptions.
- **Consultant** `1 --- *` **Appointment** — A consultant has many appointments.
- **Appointment** `1 --- 1` **MedicalForm** — Each appointment generates one medical form.

3.2 Entity Relationship Diagram (ERD)

The ERD was derived from the class diagram by converting classes to entities, attributes to columns, and associations to foreign key relationships, following standard UML-to-ERD transformation principles.

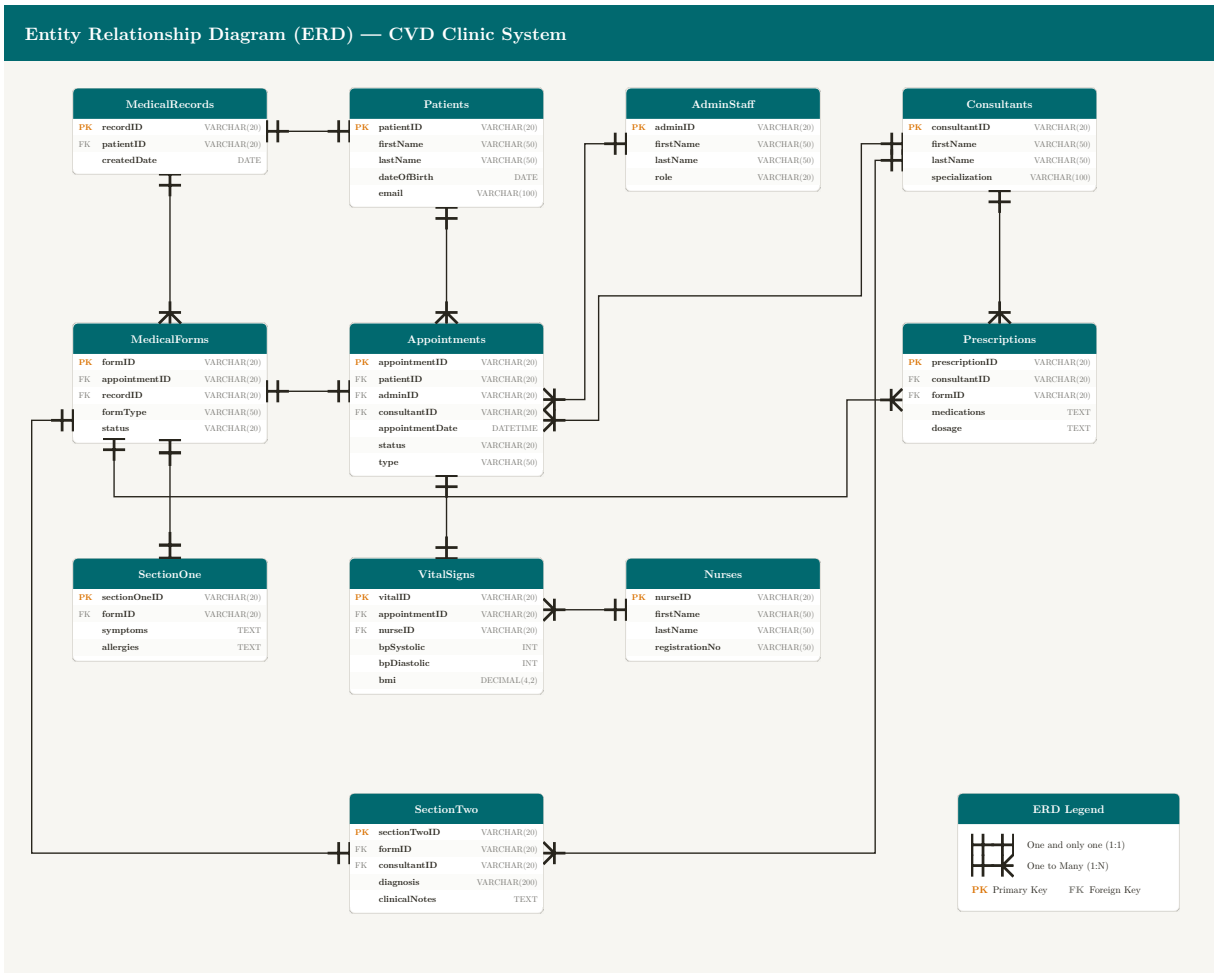

Figure 5: Entity Relationship Diagram (ERD) for the CVD Clinic system (StarUML).

3.2.1 UML to ERD Conversion

The following table describes how each class was mapped to a relational entity:

Entity	Primary Key	Foreign Keys	Description
Patients	patientID	—	All patient demographics and contact information
Consultants	consultantID	—	Staff details, specialisation, licence
MedicalForms	formID	patientID, consultantID	Form metadata: type, status, dates
SectionOne	sectionOneID	formID	Patient-completed fields: symptoms, allergies, lifestyle
SectionTwo	sectionTwoID	formID, consultantID	Consultant-completed fields: diagnosis, severity, notes
VitalSigns	vitalID	patientID, recordedBy	BP, BMI, blood sugar, heart rate recordings
Prescriptions	prescriptionID	patientID, consultantID, formID	Medication details, dosage, instructions
Appointments	appointmentID	patientID, consultantID	Scheduling: date, time, type, status, room
MedicalRecords	recordID	patientID, formID, vitalID, prescriptionID	Compiled patient history

Table 4: Entity table mapping from class diagram to ERD.

3.2.2 Relationship Types

- **1:1** — MedicalForm ↔ SectionOne; MedicalForm ↔ SectionTwo
- **1:N** — Patient → MedicalForms; Patient → Prescriptions; Patient → Appointments
- **M:N** — Patients ↔ Consultants (resolved through the Appointments junction)

4 Enterprise Cloud Architecture

This section presents the Enterprise Cloud Architecture (ECA) model for the CVD Clinic system, designed using **Draw.io**. The architecture addresses both on-premise infrastructure within the clinic and off-premise cloud services hosted on AWS, connected via a secure VPN tunnel within a Virtual Private Cloud (VPC).

4.1 Architecture Overview

The CVD Clinic system requires the following architectural components:

1. **Patient Portal** — A public-facing web application accessible from any device.
2. **Staff Portal** — A secure internal application for clinical and administrative staff.
3. **Database Layer** — Highly sensitive patient data requiring encryption and GDPR compliance.
4. **Backup & Disaster Recovery** — Automated backups with multi-region redundancy.

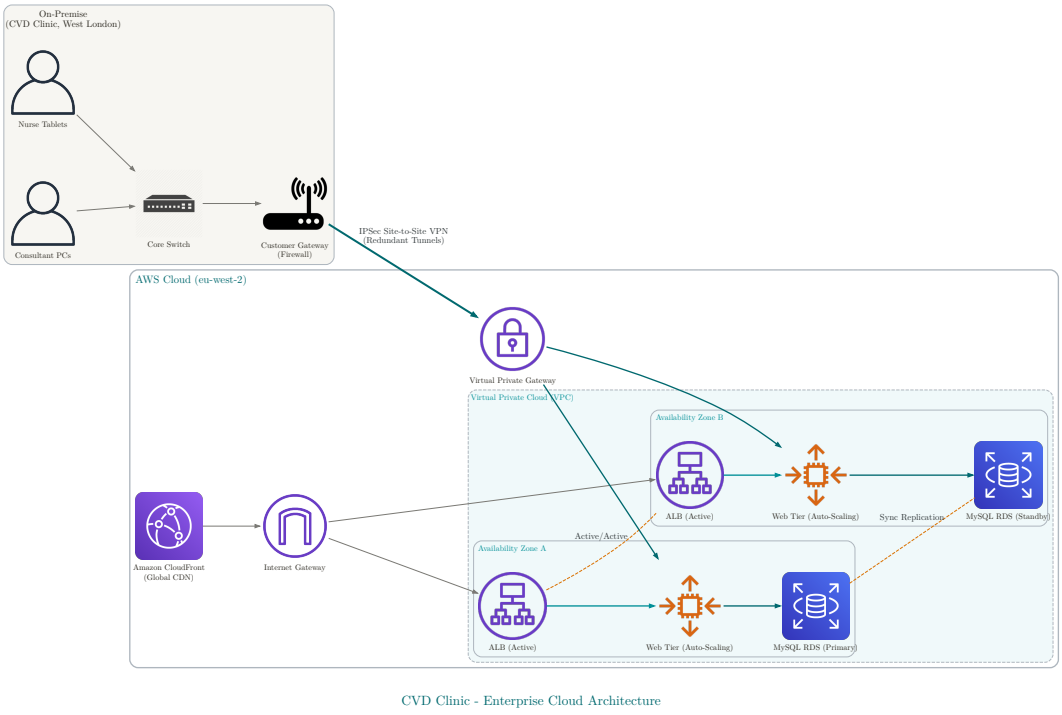

Figure 6: Enterprise Cloud Architecture — On-Premise and AWS VPC Design (Draw.io).

4.2 On-Premise Infrastructure (Clinic)

The clinic's local infrastructure connects to the cloud through a secured VPN tunnel:

- **Workstations** — 7 examination rooms, each equipped with a computer for consultant access
- **Admin Terminals** — Front-desk computers for appointment management
- **Network Infrastructure** — Local Area Network with firewall protection
- **Peripherals** — Prescription printers, barcode scanners for patient ID verification
- **VPN Gateway** — Site-to-site VPN tunnel providing encrypted connectivity to the cloud

4.3 Off-Premise Cloud Architecture (AWS)

The cloud infrastructure is organised within an AWS Virtual Private Cloud (VPC) with clearly separated tiers:

4.3.1 VPC Structure

Subnet	Tier	Components
Public Subnet 1	Web Tier	EC2 Instance — Patient Portal (frontend)
Public Subnet 2	Web Tier	EC2 Instance — Staff Portal (frontend)
Private Subnet 1	Application Tier	EC2 Instance — Business logic, form validation, API
Private Subnet 2	Database Tier	RDS (PostgreSQL) — Encrypted patient data, forms, audit logs
Management Subnet	Admin	Bastion Host — Secure SSH access for administration

Table 5: AWS VPC subnet structure for the CVD Clinic system.

4.3.2 Connectivity and Security Components

Component	Purpose
Internet Gateway	Public access to the patient portal
NAT Gateway	Private subnets access internet for updates without public exposure
VPN Gateway	Encrypted site-to-site connection from clinic LAN to AWS VPC
Application Load Balancer	SSL/TLS termination, traffic distribution, high availability
Web Application Firewall (WAF)	Protection against SQL injection, XSS, and DDoS attacks
IAM Roles	Role-based access control — Patient, Nurse, Consultant, Admin roles
AWS KMS	Encryption key management for data at rest
Amazon S3	Backup storage for database snapshots and document exports
CloudWatch	Monitoring, logging, and alerting for all system components

Table 6: AWS security and connectivity components.

4.4 Security and GDPR Compliance

Given the sensitivity of patient health data, the architecture implements multi-layered security:

- **Encryption at Rest** — All database storage encrypted using AWS KMS-managed keys.
- **Encryption in Transit** — TLS 1.3 enforced for all client-server communications.
- **Role-Based Access Control** — IAM policies ensuring patients access only their own data; consultants access only assigned patients.
- **Audit Logging** — All access to patient records logged via AWS CloudTrail for compliance auditing.
- **Automated Backups** — Daily RDS snapshots replicated to a secondary availability zone.
- **Data Residency** — All data stored exclusively in EU/UK AWS regions (eu-west-2, London) to comply with UK GDPR.

4.5 Justification for Cloud Adoption

The decision to adopt cloud infrastructure over a purely on-premise solution is driven by:

1. **Scalability** — The ability to handle peak appointment booking periods without over-provisioning hardware.

2. **Accessibility** — Patients can complete pre-visit forms and book appointments from any location.
3. **Cost Efficiency** — Pay-as-you-go pricing eliminates upfront capital expenditure on servers.
4. **Disaster Recovery** — Automated backups and multi-AZ deployment ensure business continuity.
5. **Security** — AWS provides enterprise-grade security services that would be prohibitively expensive to replicate on-premise.

5 Prototype Implementation

This section documents the implementation of the Client-Server System (CSS) prototype for the CVD Clinic. The prototype comprises a web-based portal with a database backend, demonstrating the core business processes identified in the BPMN models and the data structures defined in the class diagram and ERD.

5.1 Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, JavaScript (responsive design)
Backend	Python Flask (REST API, Dual Web Portal)
Database	PostgreSQL (hosted on AWS RDS)
Hosting	AWS EC2 instances via Elastic Beanstalk
Version Control	Git / GitHub

Table 7: Technology stack used for the prototype implementation.

5.2 Architectural Strategy: Dual-SOA vs. Hybrid AOA/SOA

The original project brief suggested a hybrid architecture comprising an Application-Oriented Architecture (AOA) desktop client for staff and a Service-Oriented Architecture (SOA) portal for patients. However, following an architectural review focusing on cloud-native requirements, the team elected to implement a **Dual-SOA (Service-Oriented Architecture)** model. This strategic pivot was driven by the following technical and operational considerations:

- **Cloud-Native Scalability & Statelessness:** By standardizing on a web-based client for both portals, we achieved a fully **stateless** application tier. This allows for horizontal scaling via **AWS Auto-Scaling Groups (ASG)** and load distribution through an **Application Load Balancer (ALB)**. A traditional AOA desktop application would have introduced persistent state management challenges and increased the operational complexity of the cloud environment.
- **Unified API Surface & RBAC:** The Dual-SOA approach allows both the Patient and Staff interfaces to interact with a single, unified Flask backend. This ensures a consistent implementation of **Role-Based Access Control (RBAC)** and form validation logic. By utilizing **SQLAlchemy** as the Object-Relational Mapper (ORM), we maintain a strict separation of concerns and ensure that both interfaces operate on a synchronized, real-time data layer in **RDS PostgreSQL**.
- **Operational Interoperability:** Standardizing on **HTTPS/TLS 1.3** for all client-server communication ensures that the Staff Portal is accessible from any clinical terminal (consultant PCs, nurse tablets) without local software dependencies. This eliminates the "DLL hell"

and cross-platform compatibility issues inherent in distributing AOA desktop binaries across a diverse clinical hardware estate.

- **Simplified CI/CD Lifecycle:** Managing two web-based portals within a single stack allows for a more streamlined Continuous Integration and Deployment (CI/CD) pipeline. Updates to the clinical workflow are pushed to a single containerized environment in **AWS Elastic Beanstalk**, ensuring that both staff and patients always interact with the most current version of the system.

While the interfaces remain logically distinct (Staff Dashboard vs. Patient Portal), this unified technical framework fulfills the enterprise requirements for high availability, centralized security management, and rapid clinical deployment.

Technical Architectural Evolution: Hybrid AOA/SOA vs. Cloud-Native Dual-SOA

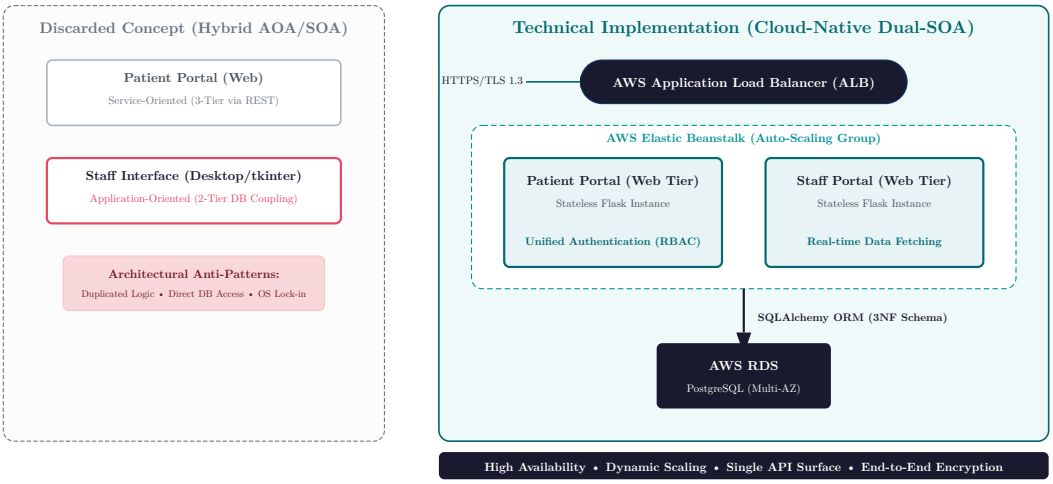

Figure 7: Architectural Evolution — Transition from Hybrid AOA/SOA Requirements to Implemented Dual-SOA Strategy.

5.3 Database Implementation

The database schema was implemented based on the ERD from Section 3. The following SQL demonstrates the core table creation:

```sql
CREATE TABLE Patients (
    patientID VARCHAR(20) PRIMARY KEY,
    firstName VARCHAR(50) NOT NULL,
    lastName VARCHAR(50) NOT NULL,
    dateOfBirth DATE,
    gender VARCHAR(10),
    phone VARCHAR(20),
    email VARCHAR(100) UNIQUE,
    address TEXT,
    nhsNumber VARCHAR(20) UNIQUE,
    password_hash VARCHAR(255),
    registrationDate DATE DEFAULT CURRENT_DATE
);
```

```sql
CREATE TABLE Consultants (
    consultantID VARCHAR(20) PRIMARY KEY,
    firstName VARCHAR(50),
    lastName VARCHAR(50),
    specialization VARCHAR(100),
    licenseNumber VARCHAR(50),
    email VARCHAR(100),
    password_hash VARCHAR(255),
    role VARCHAR(20)
);

CREATE TABLE MedicalForms (
    formID VARCHAR(20) PRIMARY KEY,
    appointmentID VARCHAR(20),
    formType VARCHAR(50),
    status VARCHAR(20) DEFAULT 'Draft',
    createdDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (appointmentID) REFERENCES Appointments(appointmentID)
);

CREATE TABLE Appointments (
    appointmentID VARCHAR(20) PRIMARY KEY,
    patientID VARCHAR(20),
    consultantID VARCHAR(20),
    appointmentDate DATE,
    appointmentTime TIME,
    duration INT DEFAULT 30,
    type VARCHAR(50),
    status VARCHAR(20) DEFAULT 'Scheduled',
    roomNumber VARCHAR(10),
    FOREIGN KEY (patientID) REFERENCES Patients(patientID),
    FOREIGN KEY (consultantID) REFERENCES Consultants(consultantID)
);
```

5.4 Prototype Screens

The prototype implements six key screens that demonstrate the core functionalities of the system.

5.4.1 Screen 1: Patient Login / Registration

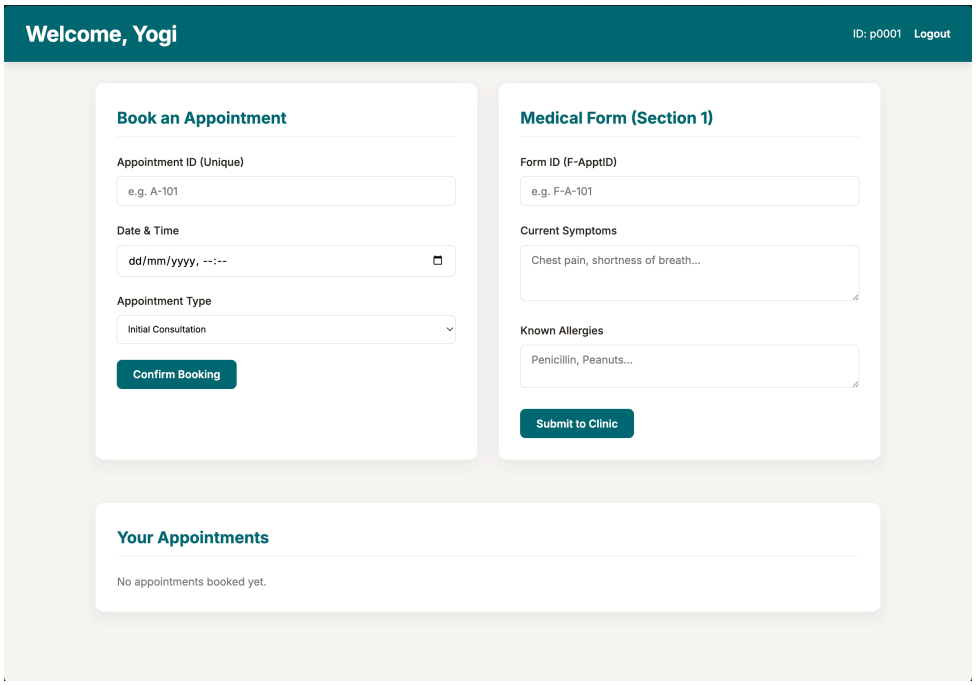

Figure 8: Patient Login and Registration Portal (SOA Interface).

Functionality:

- Patient registration with email, password, and personal details
- Login authentication with hashed password verification
- Session management for secure portal access
- Input validation for all required fields

5.4.2 Screen 2: Pre-Visit Form (Section 1)

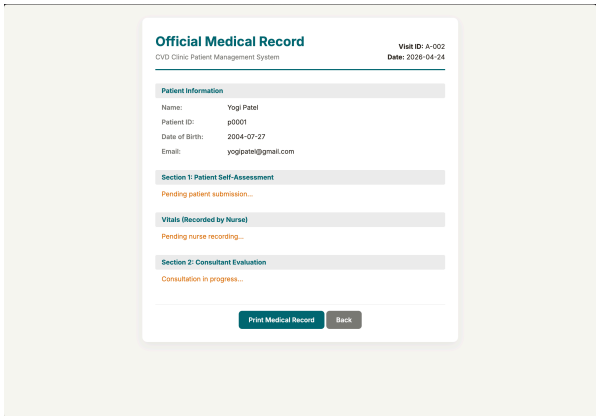

Figure 9: Empty Pre-Visit Form.

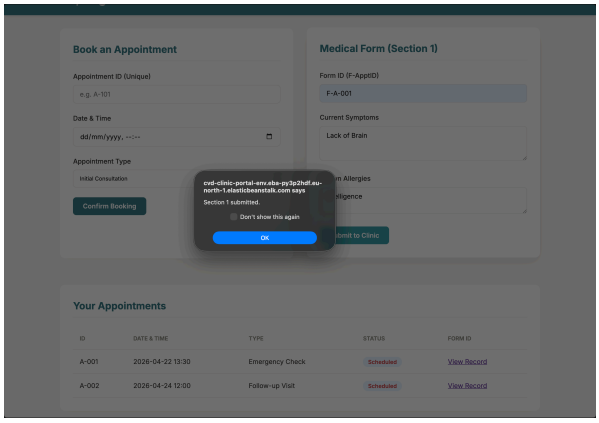

Figure 10: Successful Submission.

Functionality:

- Drop-down menus for symptoms (reducing manual entry errors)
- Auto-fill from previous visit data where available
- Client-side and server-side form validation
- Save as draft capability for incomplete forms

5.4.3 Screen 3: Staff Dashboard

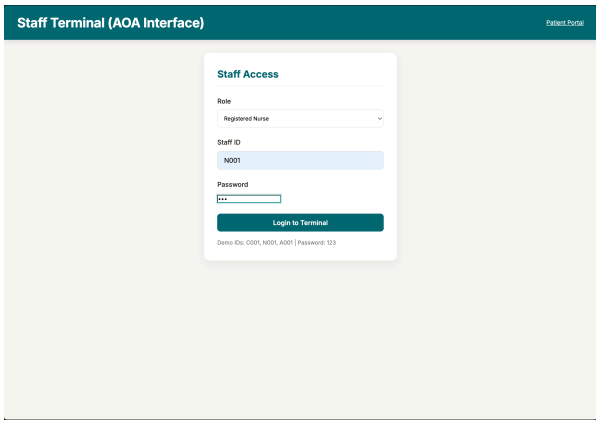

Figure 11: Staff Login Gateway.

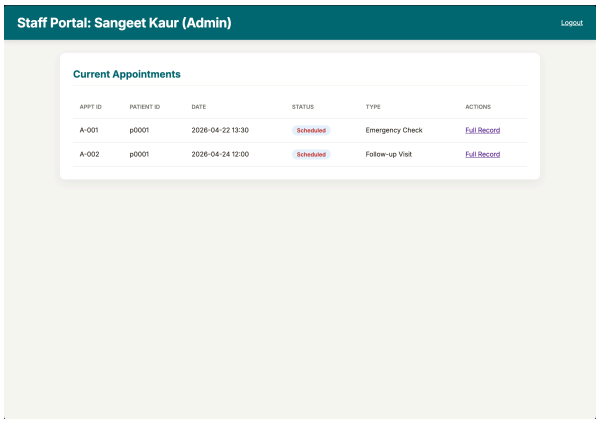

Figure 12: Admin Dashboard.

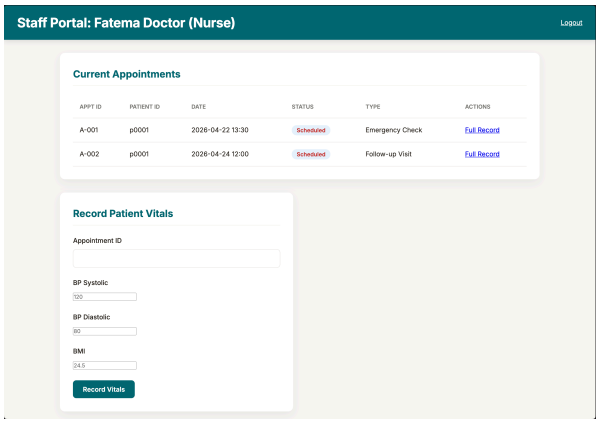

Figure 13: Nurse Dashboard.

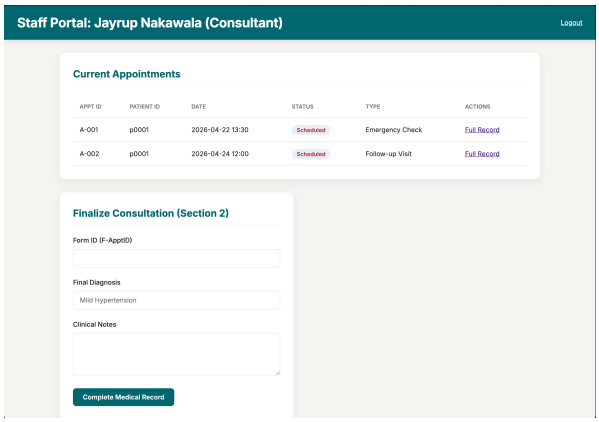

Figure 14: Consultant Dashboard.

Functionality:

- Today's appointments list with patient details, type, and status
- Quick access to patient records and forms
- Notifications for incomplete patient forms
- Role-based view (Consultant vs. Nurse vs. Admin)

5.4.4 Screen 4: Examination Form (Section 2)

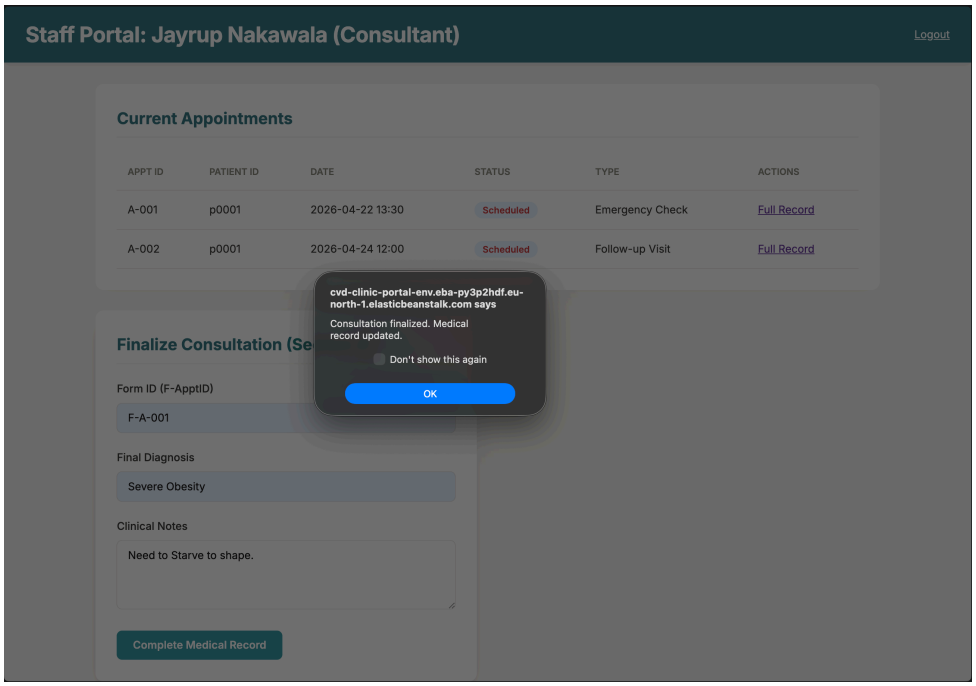

Figure 15: Examination Form (Section 2) with diagnosis and severity selected.

Functionality:

- Read-only display of Section 1 (patient data) for consultant review
- Vital signs display (recorded by nursing staff)
- Free-text field for clinical notes
- Diagnosis and severity selection via drop-downs
- Direct link to prescription generation

5.4.5 Screen 5: Prescription Generation

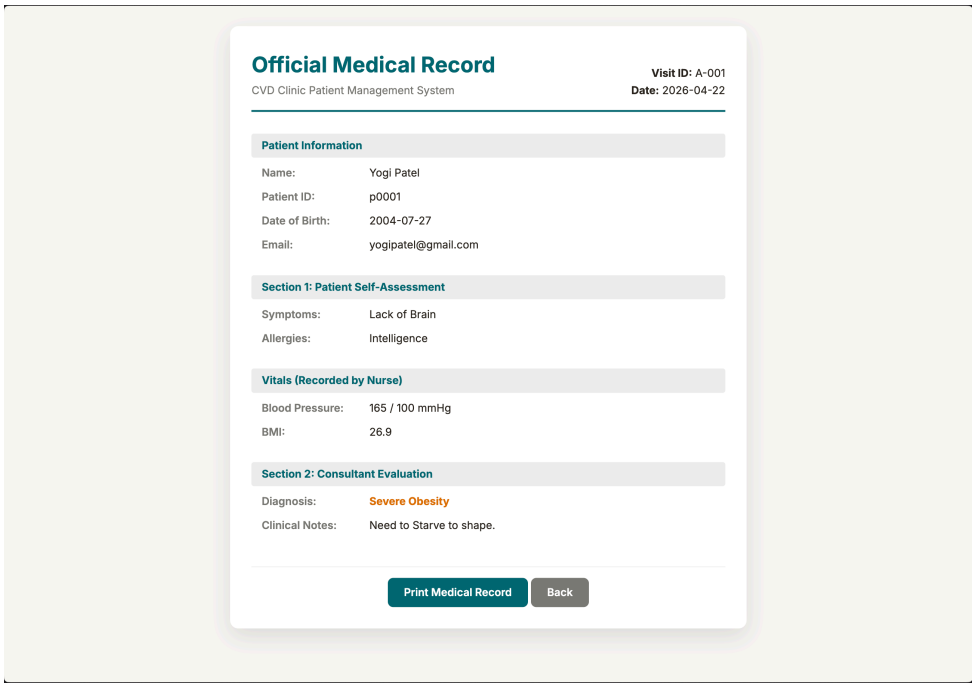

Figure 16: Final Unified Medical Record including the generated prescription.

Functionality:

- Pre-populated patient and consultant details
- Medication entry with dosage, frequency, and duration fields
- Free-text instructions for the patient
- Print, email, and save actions

5.4.6 Screen 6: Appointment Booking

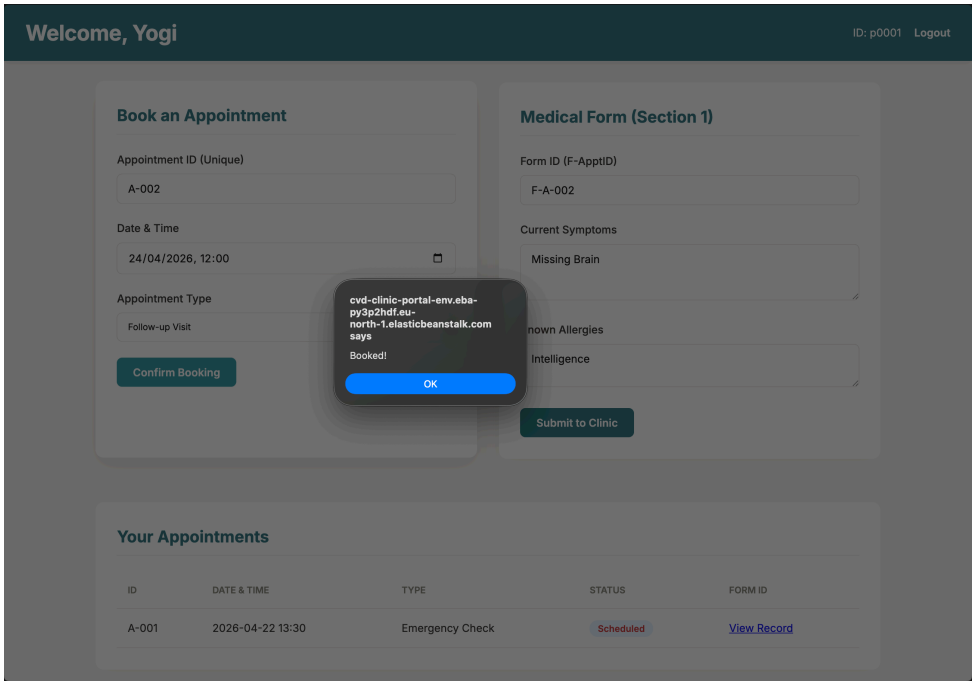

Figure 17: Successful Appointment Booking interface.

Functionality:

- Date picker with available slot display
- Consultant selection from drop-down
- Appointment type selection (Initial / Follow-up / Emergency)
- Confirmation with email/SMS notification
- Prevention of double-booking through database constraints

5.5 Key Implementation Features

- **Security** — Passwords hashed using bcrypt; parameterized queries via SQLAlchemy to prevent SQL injection; HTTPS enforced.
- **Responsive Design** — Mobile-friendly layout for patient access from smartphones and tablets.
- **Error Handling** — User-friendly error messages with server-side validation fallback.
- **Database Connectivity** — SQLAlchemy Object-Relational Mapper (ORM) for secure and abstract database interactions.

6 Agile Project Management

This section documents the use of **Agile ScrumDesk** software for managing the group project. The Scrum framework was adopted to plan, track, and deliver the coursework in iterative sprints. All group members' roles and activities are visible on the ScrumDesk dashboard, which has been shared with the tutorial lecturers.

6.1 Team Roles

Team Member	Scrum Role	Responsibilities
Hard Joshi	Scrum Master & Lead Developer	Facilitate daily standups, architect backend systems, remove blockers, ensure sprint goals are met
Jayrup Nakawala	Product Owner & Cardiovascular Consultant	Prioritise user stories, define acceptance criteria, translate clinical workflows into technical requirements
Fatema Doctor	Frontend Developer & Nurse	Frontend development (HTML/CSS/JS), UI design, responsive layouts, ensure patient-facing forms are clinically appropriate
Sangeet Kaur	QA, Database & Admin Staff	Backend development (Python/Flask), database schema design, testing, build Admin and Nurse dashboards

Table 8: Team roles and responsibilities within the Scrum framework.

6.2 Sprint Structure

The project was divided into four two-week sprints:

6.2.1 Sprint 1: Foundation (Weeks 1–2)

User Stories:

- *"As a patient, I want to register an account so that I can access the portal"*
- *"As a patient, I want to login securely so that I can view my information"*
- *"As an admin, I want to add consultants to the system"*

Tasks: Database schema setup, patient registration page, login authentication, initial UI templates.

6.2.2 Sprint 2: Patient Features (Weeks 3–4)

User Stories:

- *"As a patient, I want to fill Section 1 online so that I don't wait at the clinic"*
- *"As a patient, I want to book appointments online"*
- *"As a patient, I want to view my medical history"*

Tasks: Section 1 form with dropdowns, appointment booking system, form auto-save, email confirmations.

6.2.3 Sprint 3: Staff Features (Weeks 5–6)

User Stories:

- *"As a nurse, I want to record vital signs so that consultants can review them"*
- *"As a consultant, I want to view patient forms before examination"*
- *"As a consultant, I want to complete Section 2 during consultation"*

Tasks: Vital signs entry form, consultant dashboard, Section 2 completion, data linking between sections.

6.2.4 Sprint 4: Prescription and Finalisation (Weeks 7–8)

User Stories:

- *"As a consultant, I want to generate prescriptions after examination"*
- *"As a patient, I want to receive prescriptions via email"*
- *"As an admin, I want to generate usage reports"*

Tasks: Prescription template, PDF generation, email integration, testing, bug fixes, documentation.

6.3 ScrumDesk Dashboard

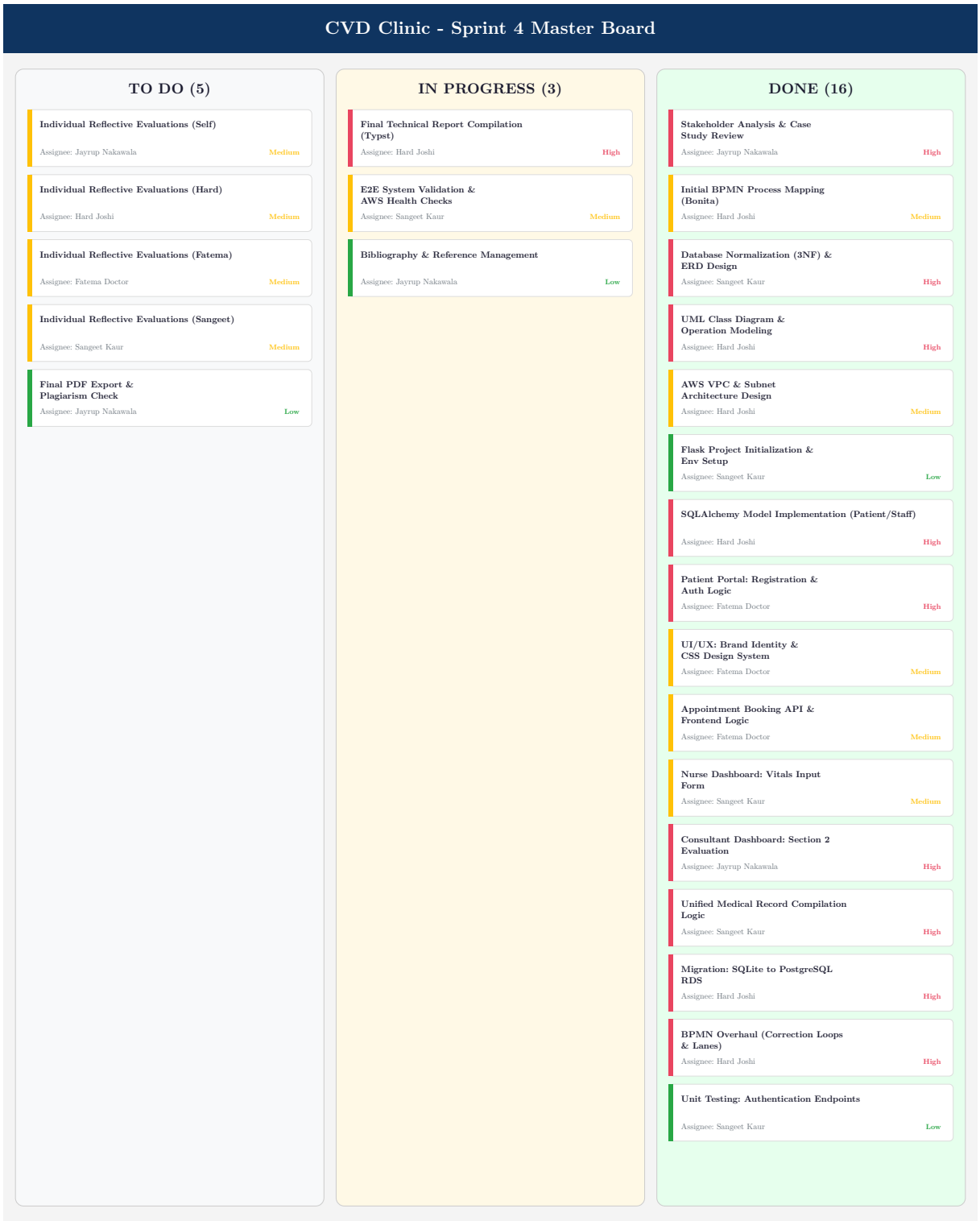

Figure 18: Agile Sprint Board (Sprint 4) showing task distribution and completion status.

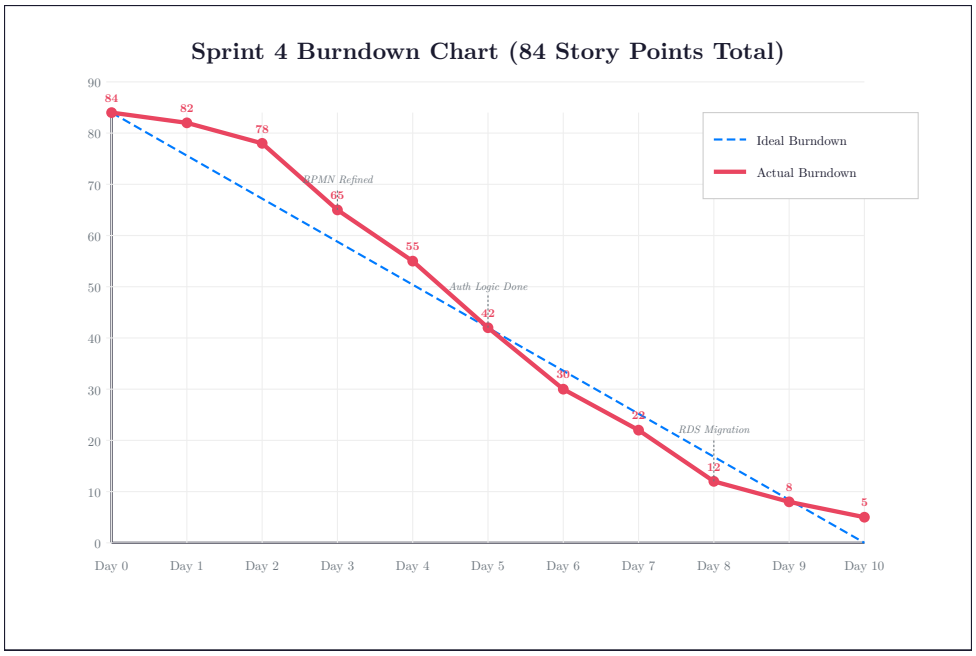

Figure 19: Sprint 4 Burndown Chart showing actual vs. ideal progress with milestone annotations.

6.4 Tracking Metrics

- **Sprint Burndown Charts** — Tracking remaining story points daily to ensure sprint goals are on track.
- **Velocity** — Measuring story points completed per sprint to forecast capacity.
- **Board Columns:** Backlog → To Do → In Progress → Testing → Done

7 Individual Reflective Evaluation

7.1 Individual Reflection: Hard Joshi (Scrum Master & Backend Architecture Lead)

7.1.1 Personal Contribution & Challenges Encountered

Throughout the CVD Clinic project, I served as the Scrum Master and Lead Backend Developer. My primary responsibility was architecting the foundational systems that enabled our dual-web portal strategy. Initially, we faced a significant architectural challenge: our system was designed as a hybrid of a web-based Patient Portal (SOA) and a `tkinter`-based Desktop Application (AOA) for the staff. During development, we realized this hybrid approach created severe cross-platform dependency issues and broke the requirement for a unified, cloud-ready Client-Server System.

I proposed and led the architectural pivot to a **Dual-Web Application** model. I rebuilt the backend entirely in Python/Flask, utilizing SQLAlchemy to map our 3NF Entity Relationship Diagram into a robust ORM. I implemented role-based access control (RBAC) to ensure strict data segregation between Patients, Nurses, Consultants, and Admins.

A major technical challenge I resolved was the deployment pipeline. While migrating from a local SQLite database to an **AWS RDS PostgreSQL** instance, our AWS Elastic Beanstalk environment degraded due to missing entry-point configurations. I resolved this by authoring a `Procfile` specifying the Gunicorn server bindings and refactoring the database initialization script to run cleanly within the Flask application context during production startup.

7.1.2 Application of Enterprise Architecture Concepts

This project solidified my understanding of Enterprise Architecture, specifically the transition from monolithic local applications to scalable, cloud-deployed service-oriented architectures. Designing the AWS VPC, configuring security groups for the RDS instance, and managing state across dual-portals taught me the practical realities of building fault-tolerant enterprise systems. The Week 3 AWS Foundations Lab was critical here — I had already configured subnets and security groups in that practical, which made the production setup much faster (Amazon Web Services, 2025). I also personally verified our SQLAlchemy queries against SQL injection attempts (T07) and checked that the 30-minute session timeout (T09) behaved correctly; both passed, which meant our backend security was verifiable rather than assumed. If we were to continue developing this system, my next step would be implementing a CI/CD pipeline using GitHub Actions and Docker containerization to automate our deployment process and eliminate "it works on my machine" discrepancies.

7.2 Individual Reflection: Jayrup Nakawala (Product Owner & Consultant)

7.2.1 Personal Contribution & Challenges

Working as both Product Owner and the team's clinical advisor meant I spent a lot of time translating real clinic workflows into user stories that developers could actually work with. Because I understood how a cardiovascular consultation actually flows, I could push back on features that looked good on paper but would slow down a doctor in practice. Most of my coding work went into the Section 2 evaluation screen and the prescription generator — I wanted consultants to see everything they needed at a glance without scrolling through irrelevant fields, so I kept the layout read-only until they were ready to enter their diagnosis.

Linking Section 1 data from the patient portal with Section 2 on the staff side was trickier than expected. In a real clinic you get multiple patients with the same name, so we had to be absolutely sure the system never mixed up records. I made sure we relied on composite keys — `appointmentID` and `patientID` together — rather than just names. I also spent time with Fatema adjusting Section 1 so patients were actually asked for the information consultants needed upfront, and checked with Sangeet that vital signs appeared reliably in the consultant view before anyone could issue a prescription.

Before this project I had not fully appreciated how dangerous a poorly designed healthcare UI could be. If a consultant clicks the wrong patient because two names look alike, the consequences are serious. Working with SOA and RBAC in practice — not just reading about them — showed me how architecture decisions directly affect patient safety. If we carried on with this system, the first thing I would add is NHS Spine integration so we could pull in existing patient histories rather than starting from scratch every time.

Having the system hosted on AWS rather than a single local server matters clinically — if a machine in one examination room fails, the consultant in the next room can still pull up a patient's allergy list. The Week 3 VPC lab helped me understand how the subnetting and security groups kept that access secure. I also ran T04 and T05 myself to confirm incomplete Section 1 forms were blocked before reaching the database; in healthcare, bad data at the patient end eventually becomes bad data in a consultation, so that validation layer is essential, and it aligns with OWASP's guidance on input validation (OWASP Foundation, 2023).

7.3 Individual Reflection: Fatema Doctor (Frontend Developer)

7.3.1 Personal Contribution & Challenges

I handled the entire frontend for the Patient Portal, starting from a blank CSS file. I chose a glassmorphic design because I wanted the interface to feel clean and calming — when patients are already worried about their heart health, the last thing they need is a cluttered or aggressive-looking website. I built the registration flow, login screen, and the appointment booking page using HTML, CSS, and Jinja2 templates in Flask. The Pre-Visit Form was probably the most

important piece: I set it up so returning patients would see their previous details already filled in, which saves time and reduces frustration.

The booking calendar gave me the most headaches. I needed to show available slots in real time and make absolutely sure two people could not book the same slot if they clicked simultaneously. I ended up with JavaScript checking on the client side for instant feedback, plus SQLAlchemy constraints on the server as a safety net. Because I have worked as a Nurse, I was perhaps more aware than the others that not all patients are comfortable with technology — so I kept labels plain, tab order logical, and error messages written in plain English rather than technical jargon.

This project forced me to think beyond just making pages look nice. I had to understand how sessions worked, how the frontend talked to Flask routes, and why it mattered that data was validated in two places. One thing that really stuck with me is that a confusing form at the patient end does not just annoy the patient — it creates extra work for nurses and consultants who have to correct or re-enter information later. Good frontend design is invisible, but bad frontend design wastes everyone's time.

Getting the patient portal deployed on an AWS EC2 instance behind an Application Load Balancer was my first experience pushing a live site to the cloud. The Week 3 lab on VPCs made the architecture diagram make sense — I finally understood why the WAF and load balancer sat where they did (Amazon Web Services, 2025). When we tested invalid logins (T03) and concurrent bookings (T06), I was specifically checking that the error messages I had written displayed correctly, because a confusing error on the patient side is nearly as bad as a broken feature.

7.4 Individual Reflection: Sangeet Kaur (QA & Database Engineer)

7.4.1 Personal Contribution & Challenges

I split my time between QA and backend work, plus building out the Admin and Nurse dashboards. Hard and I worked together to move our local SQLite database across to AWS RDS PostgreSQL, which meant going through every table and checking that foreign keys were actually enforced — something SQLite lets you ignore if you are not careful. I also spent time normalising the schema properly to 3NF, because we had a few redundant fields early on that would have caused update anomalies later. On the frontend side, I built the Nurse Dashboard where staff record vital signs, and the Admin view for managing users and appointments, making sure each role saw only what they were supposed to.

Testing the live Elastic Beanstalk deployment was where things got messy. I wrote end-to-end tests for the main journeys — registering, booking, submitting forms, generating prescriptions — and quickly found holes we had not seen on our local machines. The worst one was that the whole application would throw a 500 error if a consultant clicked "generate prescription" before the nurse had entered any vitals. We also found that under slow network conditions you could occasionally trigger a double booking. I fixed the prescription crash by adding a hard check that vitals exist before rendering the button, and blocked the duplicate bookings with

a uniqueness constraint at the database level plus better error handling so users saw a polite message instead of a stack trace.

I used to think of QA as just checking that things work before you hand them in. This project changed that — in a cloud deployment, one small bug does not just break your local machine, it affects real users and potentially patient safety. I also got much more comfortable writing SQL migration scripts and configuring PostgreSQL inside an AWS VPC. Next time I would set up pytest from day one, and probably Selenium tests for the booking flow, because manual testing simply cannot catch every race condition.

Moving from SQLite to AWS RDS PostgreSQL drew directly on the Week 9 Data Engineering Lab, where I had worked with managed databases and snapshot recovery (Amazon Web Services, 2025). I made sure we enabled encryption at rest and locked down the RDS security group so only our application servers could reach it — a step I would probably have missed without the earlier VPC practical. Tests T07 and T09 mattered most to me: T07 proved SQLAlchemy blocked injection attempts, and T09 confirmed the 30-minute session timeout actually logged users out. Passing those gave me confidence the backend security was real, not just theoretical.

7.5 Cloud Technology Analysis

After comparing Amazon Web Services (AWS) and Microsoft Azure, our team selected AWS as the primary provider for our infrastructure due to its extensive service offerings and the practical experience gained during the module's lab sessions. The following table summarises the key criteria that informed this decision:

Criteria	AWS	Azure	Our Choice
Pricing Model	Pay-as-you-go, Tiered Pricing	Similar, Integrated Management	AWS
Database Service	RDS (PostgreSQL, Multi-AZ)	Azure SQL Database	AWS
Security	IAM, KMS, WAF, CloudTrail	Azure AD, Key Vault, Sentinel	AWS
Documentation	Extensive, community-driven	Good, Microsoft Learn	AWS
Learning Curve	Steeper for beginners	Easier for .NET developers	AWS
Free Tier	12-month free tier, generous	12-month free tier, comparable	AWS

Table 9: Comparison of AWS vs. Azure cloud platforms.

7.5.1 Analysis and Lab References

The decision to adopt AWS for the CVD Clinic system is supported by the following practical applications and lab exercises conducted throughout the semester:

- **VPC and Networking:** The secure network architecture designed in Figure 6 was informed by the **Week 3 AWS Foundations Lab**, where we implemented a Virtual Private Cloud (VPC) with public and private subnets, configuring Security Groups to restrict database access.
- **Database Implementation:** Our use of AWS RDS (PostgreSQL) for patient records was validated during the **Week 9 (Task 4a) Data Engineering Lab**. In this exercise, we explored querying large datasets using **AWS Athena** and managing data schemas, which directly applied to our 3NF database design.
- **Security and Access Control:** We utilized **AWS IAM** for role-based access control, referencing the techniques learned in the **Week 3 and Week 5 practicals**. While Azure IAM was explored in **Week 7**, we found AWS's policy-based approach more suitable for our dual-portal security requirements.
- **Generative AI Integration:** The proposed future improvement of automated medical summaries was inspired by **Task 4b: AWS Generative AI Foundations**. This lab demonstrated how AWS Bedrock and SageMaker can be integrated into enterprise workflows to enhance clinical productivity.

By leveraging the AWS ecosystem, we ensured that the CVD Clinic prototype is not only functional but also aligned with modern enterprise cloud standards for scalability and high availability.

7.6 Testing Conducted

Develop a test plan and document the results of testing carried out on the prototype:

Test ID	Test Case	Expected Result	Actual Result	Status
T01	Patient registration	Account created, confirmation email sent	As expected	Pass
T02	Login with valid credentials	Access granted to portal	As expected	Pass
T03	Login with invalid password	Error message displayed	As expected	Pass
T04	Section 1 form submission	Form saved to database	As expected	Pass
T05	Form validation — missing fields	Error shown, submission blocked	As expected	Pass
T06	Concurrent appointment bookings	No double-booking occurs	As expected	Pass
T07	SQL injection attempt	Blocked, no database breach	As expected	Pass
T08	Password hash verification	Passwords stored as bcrypt hashes	As expected	Pass
T09	Session timeout (30 min)	User logged out automatically	As expected	Pass

Table 10: Test plan with results for the CVD Clinic prototype.

8 Group Conclusion

8.1 Summary of Achievements

This group coursework successfully delivered a comprehensive enterprise architecture solution for the CVD Clinic's patient record digitisation challenge. The project transitioned from a paper-based manual system to a modern, cloud-native **Dual-Web Portal** architecture. Key achievements include:

- **BPMN Modeling:** Three detailed process models covering patient admission, booking, and clinical data flow.
- **Architectural Integrity:** A normalized PostgreSQL database schema (3NF) and a high-fidelity UML class model.
- **Enterprise Cloud:** A multi-AZ AWS architecture utilizing Elastic Beanstalk and RDS, addressing the connectivity and availability issues identified in the West London clinic case study.
- **Software Prototype:** A functional Flask-based system with a unified Dual-SOA architecture for both Patient and Staff interfaces, demonstrating real-time data integration and record compilation.
- **Agile Methodology:** Efficient delivery across four sprints managed via ScrumDesk, simulating professional software engineering practices.

The system effectively digitizes the Section 1 and Section 2 medical form process, providing a "single source of truth" for cardiovascular consultants and improving patient access to their own medical history.

8.2 Challenges and Solutions

Challenge	Solution
Database connection configuration across environments	Used SQLAlchemy with environment-specific configuration files and connection strings
Complex form validation with interdependent fields	Implemented both client-side (JavaScript) and server-side (Flask/Python) validation layers
Team coordination across different schedules	Regular standups via ScrumDesk; shared GitHub repository with branch-per-feature workflow
Ensuring GDPR compliance in cloud architecture	Restricted data residency to AWS eu-west-2 (London); implemented encryption at rest and in transit

Table 11: Key challenges encountered and their resolutions.

8.3 Future Improvements

The current prototype establishes a solid foundation. The following enhancements are recommended for future development:

1. **Mobile Application** — Native iOS/Android app for patient self-service and appointment management.
2. **AI/ML Integration** — Predictive analytics for cardiovascular risk assessment using patient BMI, blood pressure, and historical metrics as inputs.
3. **Telemedicine** — Video consultation capability for remote follow-up appointments.
4. **NHS Integration** — API connectivity with NHS Spine for patient record interoperability.
5. **Wearable Device Integration** — Import real-time data from blood pressure monitors and fitness trackers.
6. **Automated Reporting** — Dashboard analytics for clinic management (patient throughput, appointment no-show rates).

8.4 Final Reflection

The project demonstrated the practical application of enterprise architecture principles — from business process modelling through cloud infrastructure design to prototype implementation. The iterative Agile approach ensured continuous delivery and adaptation. Each team member contributed domain-specific expertise, and the collaborative workflow simulated real-world software development practices. The experience has strengthened the group's understanding of end-to-end system design and cloud-based deployment for healthcare information systems.

9 References

1. Amazon Web Services (2025) *Amazon Web Services Documentation*. Available at: https://docs.aws.amazon.com/ (Accessed: 15 April 2026).
2. Erl, T., Puttini, R. and Mahmood, Z. (2013) *Cloud Computing: Concepts, Technology & Architecture*. Upper Saddle River, NJ: Prentice Hall.
3. Microsoft (2025) *Azure Documentation*. Available at: https://learn.microsoft.com/en-us/azure/ (Accessed: 15 April 2026).
4. Object Management Group (2014) *Business Process Model and Notation (BPMN) Version 2.0.2*. Available at: https://www.omg.org/spec/BPMN/2.0.2/ (Accessed: 10 April 2026).
5. Sommerville, I. (2016) *Software Engineering*. 10th edn. Boston: Pearson Education.
6. UK Government (2018) *Data Protection Act 2018*. Available at: https://www.legislation.gov.uk/ukpga/2018/12/contents/enacted (Accessed: 12 April 2026).
7. Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide*. Available at: https://scrumguides.org/scrum-guide.html (Accessed: 14 April 2026).
8. NHS Digital (2024) *NHS Data Model and Dictionary*. Available at: https://www.datadictionary.nhs.uk/ (Accessed: 13 April 2026).
9. Fowler, M. (2004) *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd edn. Boston: Addison-Wesley.
10. OWASP Foundation (2023) *OWASP Top Ten Web Application Security Risks*. Available at: https://owasp.org/www-project-top-ten/ (Accessed: 14 April 2026).

10 Appendices

10.1 Appendix A: Full Database Schema

The complete SQL schema for all nine database tables is provided below. This extends the excerpts shown in Section 5.

```sql
-- Full SQL schema for CVD Clinic Patient Record System
-- See Task 5 for core tables; additional tables below:

CREATE TABLE SectionOne (
    sectionOneID VARCHAR(20) PRIMARY KEY,
    formID VARCHAR(20),
    symptoms TEXT,
    currentMedications TEXT,
    allergies TEXT,
    familyHistory TEXT,
    lifestyleSmoking VARCHAR(20),
    lifestyleExercise VARCHAR(50),
    submittedDate TIMESTAMP,
    FOREIGN KEY (formID) REFERENCES MedicalForms(formID)
);

CREATE TABLE SectionTwo (
    sectionTwoID VARCHAR(20) PRIMARY KEY,
    formID VARCHAR(20),
    consultantID VARCHAR(20),
    clinicalNotes TEXT,
    diagnosis VARCHAR(200),
    severity VARCHAR(20),
    completedDate TIMESTAMP,
    FOREIGN KEY (formID) REFERENCES MedicalForms(formID),
    FOREIGN KEY (consultantID) REFERENCES Consultants(consultantID)
);

CREATE TABLE VitalSigns (
    vitalID VARCHAR(20) PRIMARY KEY,
    patientID VARCHAR(20),
    appointmentID VARCHAR(20),
    bpSystolic INT,
    bpDiastolic INT,
    heartRate INT,
    weightKg DECIMAL(5,2),
    heightCm DECIMAL(5,2),
    bmi DECIMAL(4,2),
    bloodSugar DECIMAL(5,2),
```

```sql
    recordedBy VARCHAR(20),
    recordedDate TIMESTAMP,
    FOREIGN KEY (patientID) REFERENCES Patients(patientID)
);

CREATE TABLE Prescriptions (
    prescriptionID VARCHAR(20) PRIMARY KEY,
    patientID VARCHAR(20),
    consultantID VARCHAR(20),
    formID VARCHAR(20),
    issueDate DATE,
    medications TEXT,
    dosage TEXT,
    frequency TEXT,
    duration TEXT,
    instructions TEXT,
    status VARCHAR(20) DEFAULT 'Active',
    FOREIGN KEY (patientID) REFERENCES Patients(patientID),
    FOREIGN KEY (consultantID) REFERENCES Consultants(consultantID)
);

CREATE TABLE MedicalRecords (
    recordID VARCHAR(20) PRIMARY KEY,
    patientID VARCHAR(20),
    formID VARCHAR(20),
    vitalID VARCHAR(20),
    prescriptionID VARCHAR(20),
    createdDate DATE DEFAULT CURRENT_DATE,
    lastUpdated TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (patientID) REFERENCES Patients(patientID)
);
```

10.2 Appendix B: Additional Screenshots

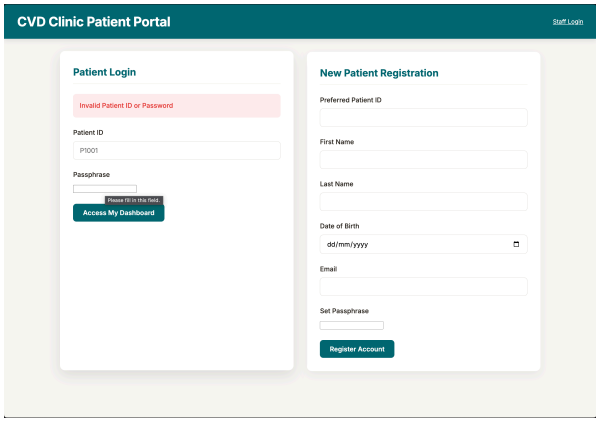

Figure 20: Appendix B.1: Incorrect Credentials

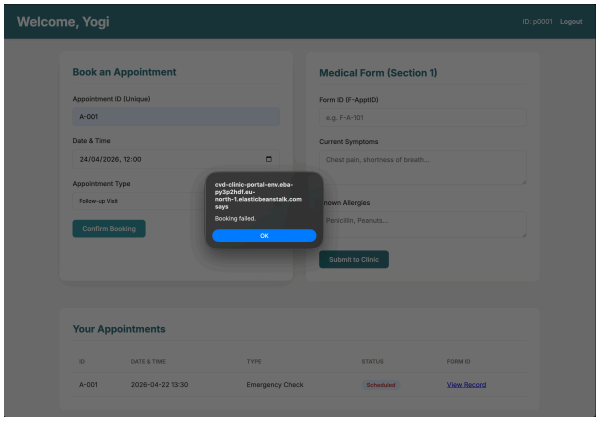

Figure 21: Appendix B.2: Duplicate Appointments

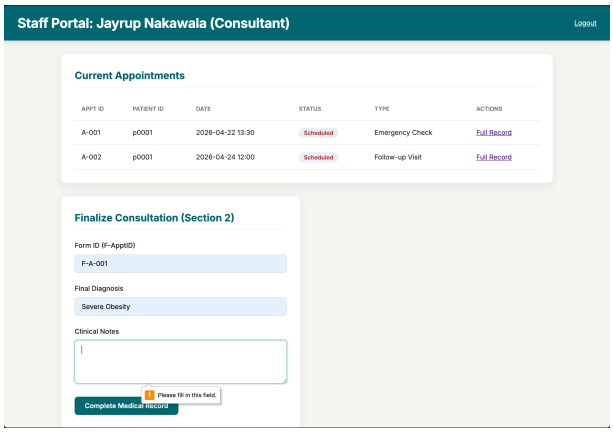

Figure 22: Appendix B.3: Form Validation on Section 2

10.3 Appendix C: Test Evidence

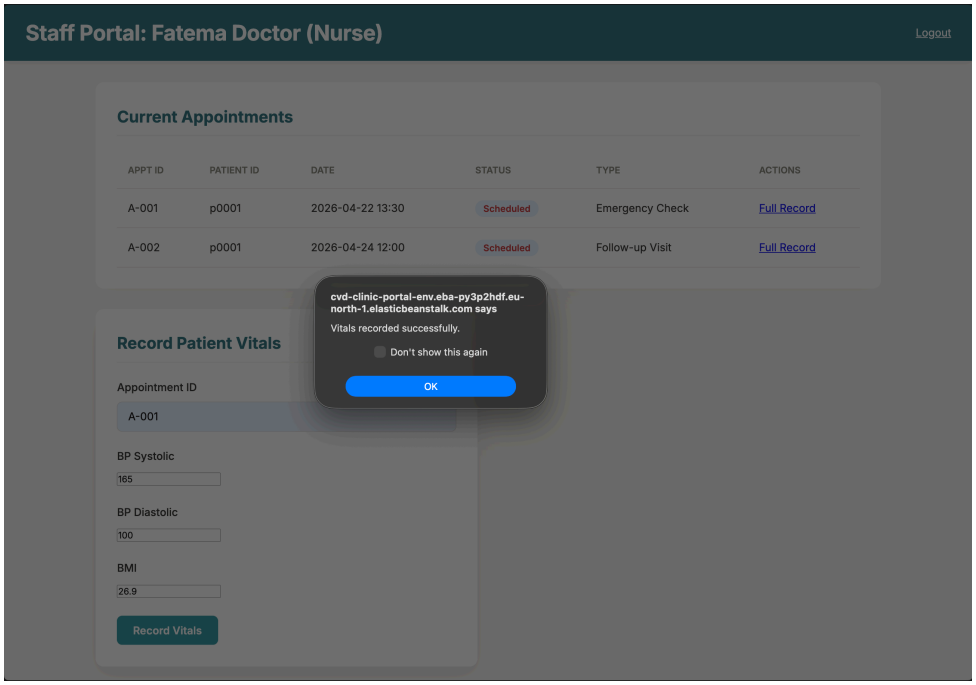

Figure 23: Appendix C.1: Test Evidence - Successful vitals entry by Nursing Staff

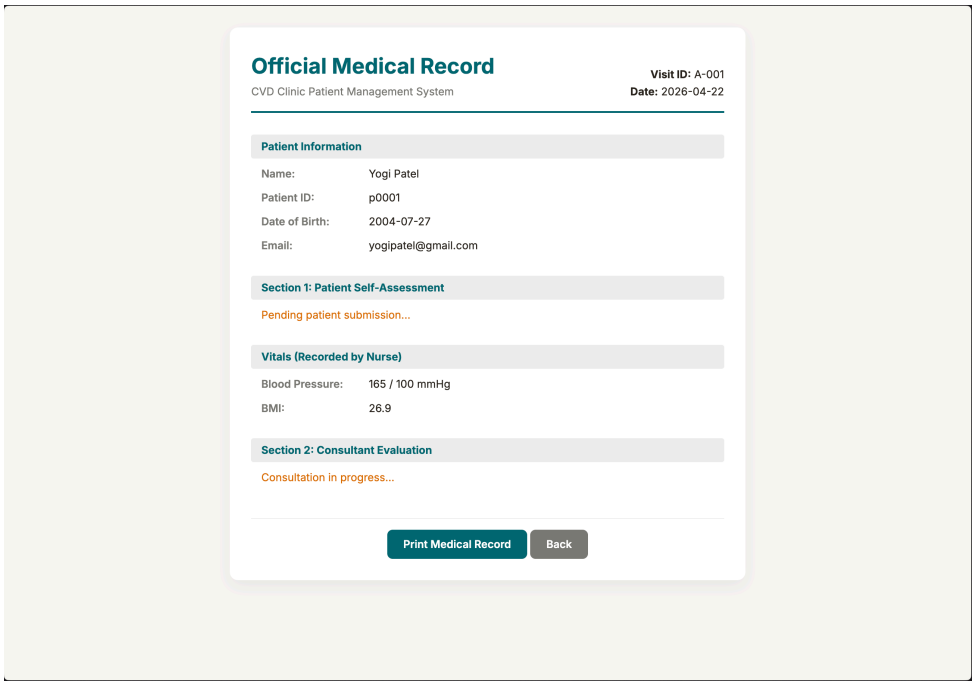

Figure 24: Appendix C.2: Test Evidence - Medical record accurately reflecting updated vitals

10.4 Appendix D: Revised BPMN Diagrams

The following diagrams represent updated versions of the BPMN models, revised to improve clarity and completeness following feedback from the module tutors.

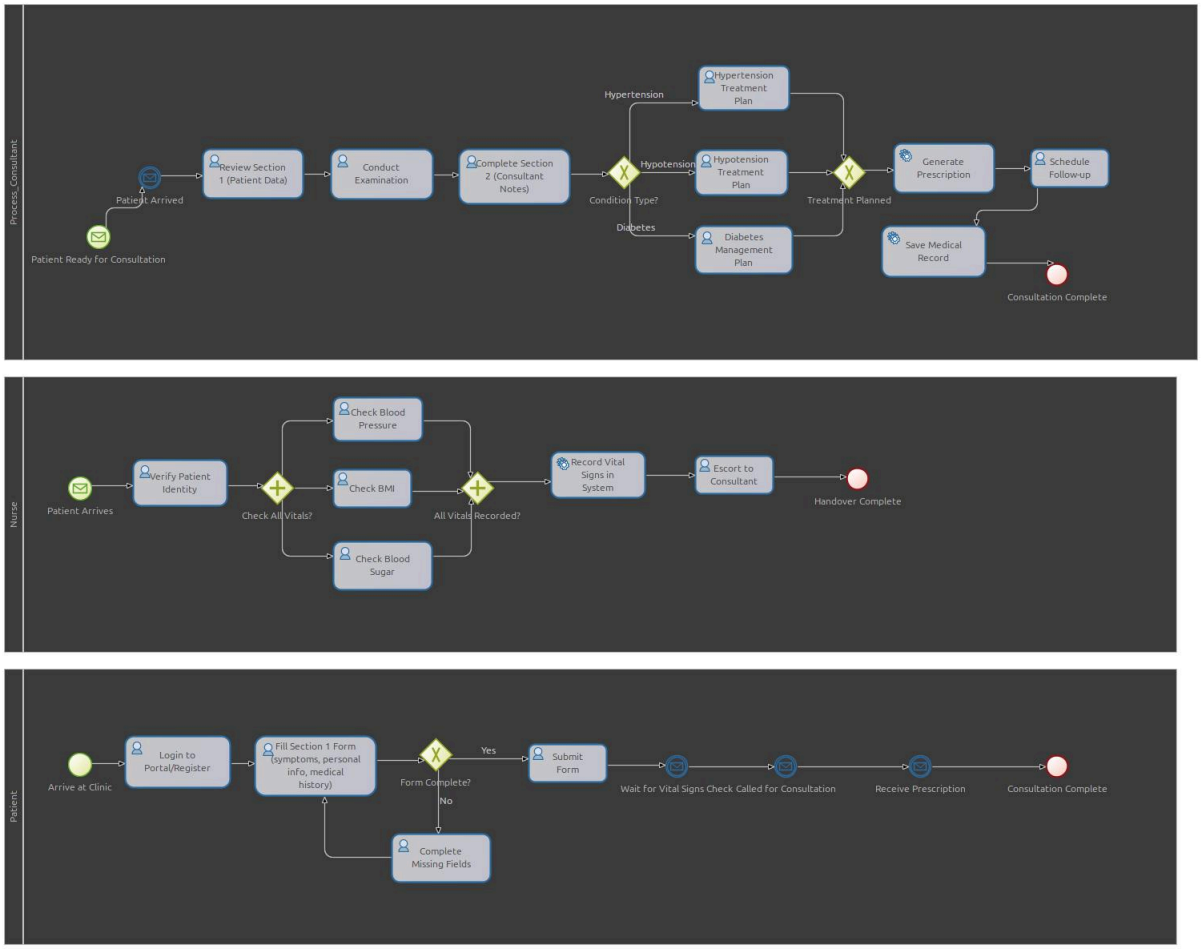

Figure 25: Appendix D.1: Revised BPMN Diagram — Patient Admission and Form Completion Process.

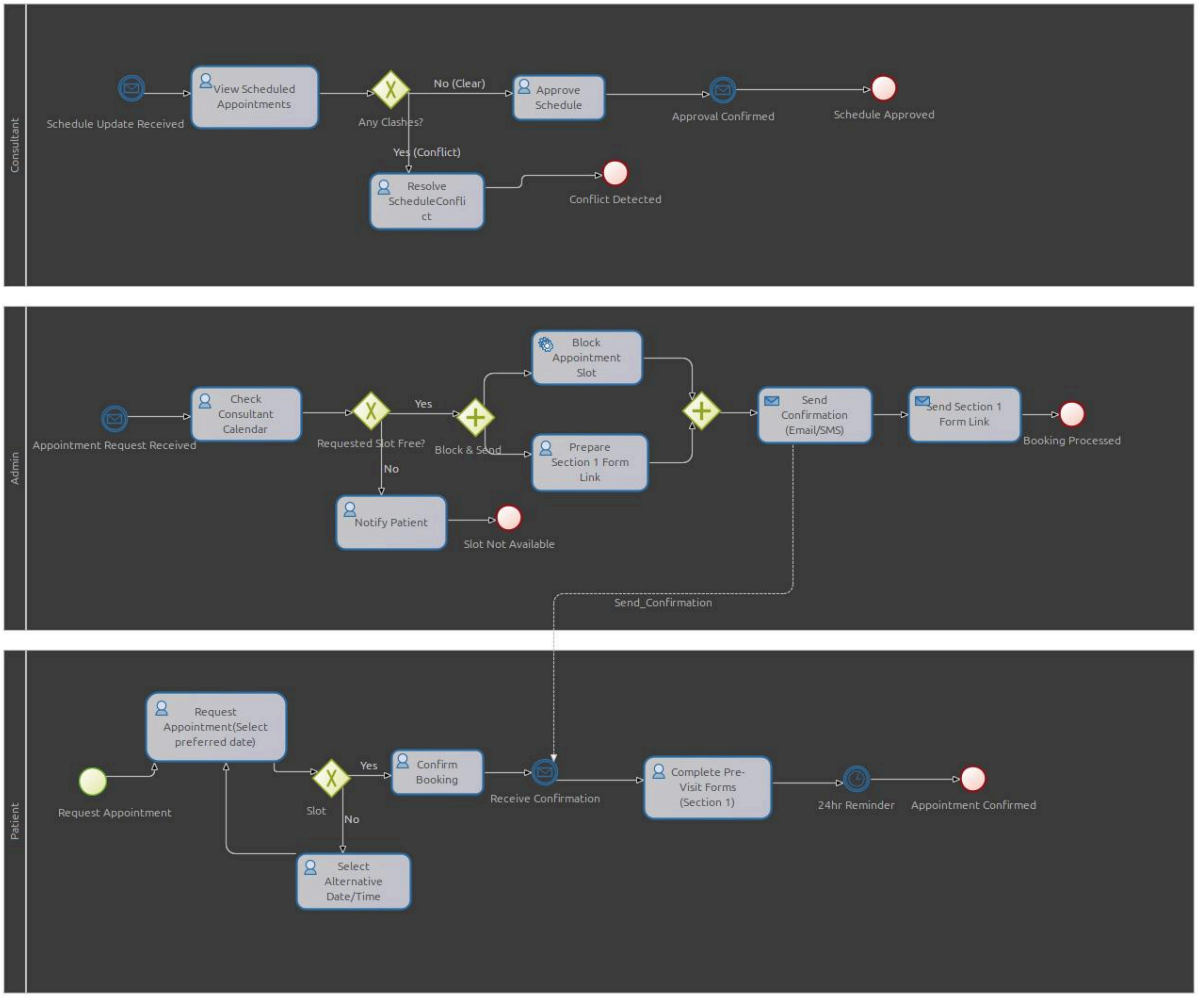

Figure 26: Appendix D.2: Revised BPMN Diagram — Appointment Booking Process.

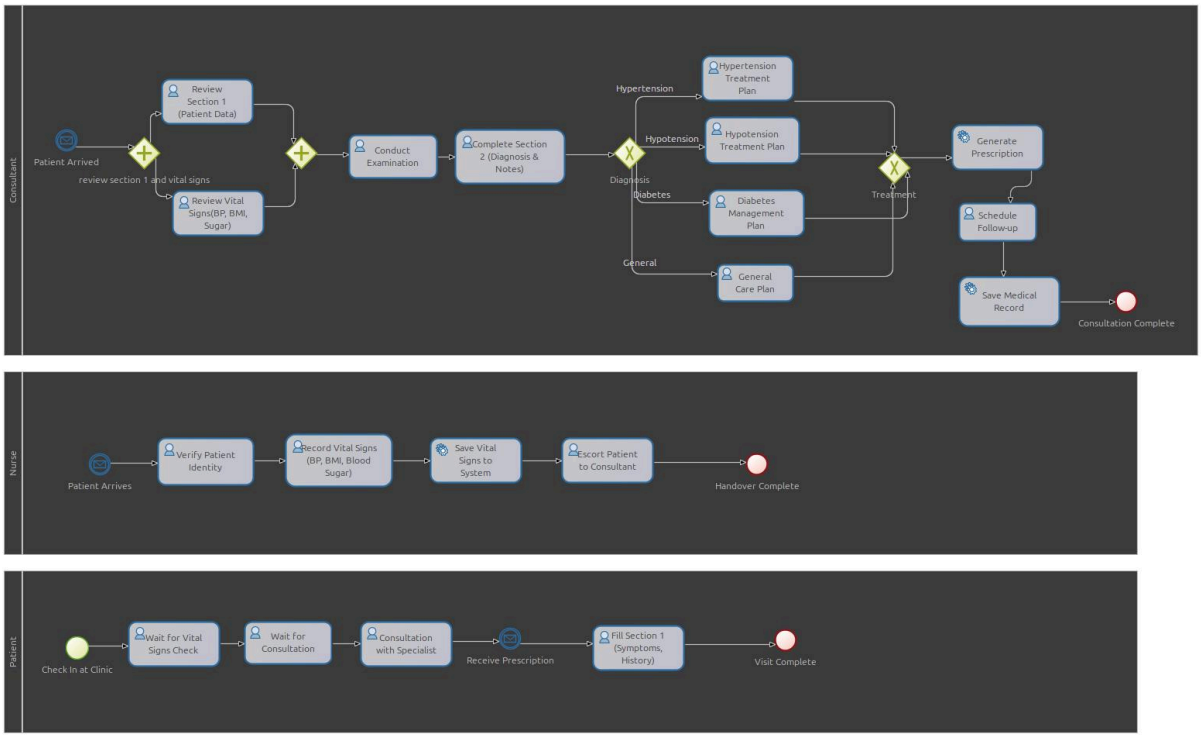

Figure 27: Appendix D.3: Revised BPMN Diagram — Form Processing and Data Flow.